Machine Translated by Google

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES TOULOUSE
DEPARTEMENT DE GENIE ELECTRIQUE ET INFORMATIQUE

4th year IR-SI

Academic Year 2020-2021

Computer Systems Project

From compiler to microprocessor

Daniela Dragomirescu
Eric Alata

Machine Translated by Google

Project organization:

1. Development of a compiler using LEX and YACC

ÿ Application of Automata & Languages

ÿ Presentation of the LEX and YACC software

2. Design of a RISC-type microprocessor with pipeline

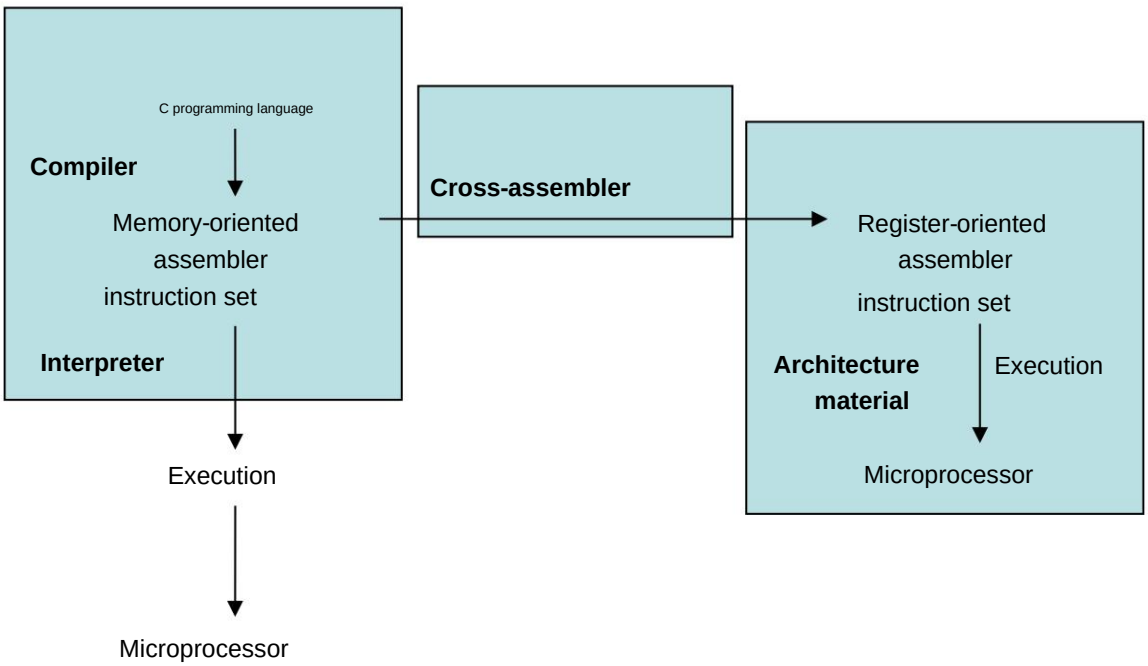
ÿ Application of hardware architectures for computer systems

Machine Translated by Google

Objective

The objective of this Computer Systems Project is to create a complete computer system.
We will create a compiler that translates a source language (similar to C) into a target language (a memory-oriented assembly instruction set). Creating a microprocessor corresponding to a memory-oriented instruction set is difficult. However, creating a
A microprocessor corresponding to a register-oriented instruction set is easier to implement. A **cross-compiler** to translate this memory-oriented instruction set into a register-oriented instruction set will therefore be necessary.

Here is the complete diagram of our computer system:



Machine Translated by Google

Developing a compiler using LEX and YACC

1. OBJECTIVE

The objective of this practical exercise is to create a compiler for a version simplified C language using the LEX lexical analyzer generator and the generator YACC (Yet Another Compiler) parser .

The practical work will be structured in 4 parts:

1. Part One – You will develop a compiler for a language that allows the Manipulating C-type arithmetic expressions. The compiler will generate the code corresponding to the assembly language. Next, you will write an interpreter for the language. assembly language will be used to execute the code. Finally, you will create a test program. of the compiler and interpreter developed.

An interpreter will be provided for the 2020-2021 academic year – COVID pandemic relief

2. Second part – you will add to your C-like language the implementation of conditional expressions **if** and **while .**

3. Third part – you will add pointer and/or functions and/or error handling (with varying degrees of difficulty). **This part is Optional for the 2020-2021 academic year - COVID pandemic relief. The Completing this section will lead to grades above 16.**

4. Cross-assembler – **The cross-assembler will be provided for the 2020 academic year. 2021**

• Development of a compiler for a simplified C-like language and of the interpreter of an assembly language

The input language is a C-like language that recognizes:

• the **main() function,** the curly braces **{ and }**

• Integer constants (keyword: **const)** and their names

Machine Translated by Google

- Integer variables (keyword: **int)** and their names. The name of a variable or a The constant must begin with a letter and can contain **letters,** numbers , and the underscore _
- arithmetic operations defined by the following mathematical symbols: +, -, *, /, **=** and parentheses **( )**
- Separators: space, TAB, and comma
- the end of line \n
- the end of an instruction signaled by " **;** " as in classic C programming
- The printf() function has only one parameter: the variable whose value should be displayed
- Integers must be recognized in both decimal and exponential form
- Our C language allows the declaration of multiple variables on the same line

(e.g., `int toto, mimi=5;`) or the declaration of several constants on the same line. In this In this case, don't forget to allocate memory for each of the declared variables and/or constants.

- Our C language distinguishes between lowercase and uppercase letters. The keywords of Languages will only be recognized if they are written in lowercase.
- Constants and variables can only be declared within the function. main(). This declarative part must be placed before the instructions, without fail.
- no other function calls are recognized by our simplified C language, in a first phase

First, we will create the lexical analyzer that detects typed words. (tokens) specific to our language (as described above) and displays a message on the screen which indicates the tokenized word read. Once the lexicographical analyzer has correctly Once developed, we will modify the lexical specification in order to pass these typed words to YACC.

In the second step we will create the parser which analyzes our C-type language and produces the corresponding assembler code as output.

Description of the assembly language to be used:

The data is stored in an array- type **data memory ,** with **a The first area is reserved for declared variables and constants ,** and **the second area for temporary results** (see course section 5.3.4 of chapter 3). The release of memory space.

Machine Translated by Google

The known instructions are:

- addition of the 2 operands:

ADD @result @operand1 @operand2

operation code ADD 1

example :

if operand 1 is at address 50, operand 2 is at address 55 and the result at address 250 we generate:

1 250 50 55

This coding will allow for easier interpretation.

- multiplication of the 2 operands:

MUL @result @operand1 @operand2

MUL 2 operation code

- subtraction

SOU @result @operand1 @operand2

operation code SOU 3

- division

DIV @result @operand1 @operand2

DIV operation code 4

- copy

COP @result @operand

COP 5 operation code

- assignment

AFC @constant value result

AFC operation code 6

- unconditional jump

JMP instruction number

JMP 7 operation code

- skip if the condition is false

JMF @X instruction number

JMF operation code 8

An integer code of **1** is assigned for a **true** result and **0** for a **false result.**

- comparison

lower :

Machine Translated by Google

INF @result @operand1 @operand2

operation code INF 9

superior :

SUP @result @operand1 @operand2

operation code SUP A

equal :

EQU @result @operand1 @operand2

operation code EQU B

- print

PRI @result

PRIC operation code C

After your parser outputs, you will write the code instructions. the corresponding assembler in plain text in a first file, as well as the encoded version in a second file.

Thirdly, you will develop the assembler language interpreter **by using LEX and YACC** (from the previously generated encoded version). This interpreter will display On the screen is the result of our program written in C language and which uses the printf() function.

Example :

```
hand()
{ int i, j,k ,r;
i = 3;
j=4;
k=8;
printf (i) ;
r=(i+j)*(i+k/j) ;
printf(r);
}
```

The result displayed by the interpreter on the screen will be:

```
3
35
```

So, as a final step you will test the developed compiler and interpreter.

Machine Translated by Google

• Implementation of conditional expressions for if and while .

You will add conditional expressions as well as if and while statements with the same syntax as the C language. You will make the necessary modifications to level of your compiler's lexical and syntactic analyzer.

• Implementation of pointers - optional

You are going to add the implementation of pointers in the C language. You are going to do the necessary modifications to the lexical and syntactic analyzer of your compiler. You will also add the necessary assembler instructions for the implementation of pointers.

• Function implementation - optional

You will add the implementation of functions in the C language. You will do the necessary modifications to the lexical and syntactic analyzer of your compiler. You will also add the necessary assembler instructions for the implementation of functions.

• Error handling - optional

Initially, a simple error handling system needs to be implemented at the level of the compiler. It will detect when an error has occurred and display a message. error and the line on which the error is initially located.

Secondly, it is necessary to be able to analyze an error in order to then continue the analysis of the input string.

Next we will present the two software programs used: LEX and YACC.

Machine Translated by Google

2. LEX: Lexical analyzer generator

2.1 Introduction

LEX is a tool that, starting from the description of the lexicography of a language (data input) generates an associated lexicographic analyzer. This generated analyzer is a program C. The user specifications provided as LEX data are converted into a a program that recognizes a series of terminals (tokens) and also executes fragments of code (actions) also provided by the user.

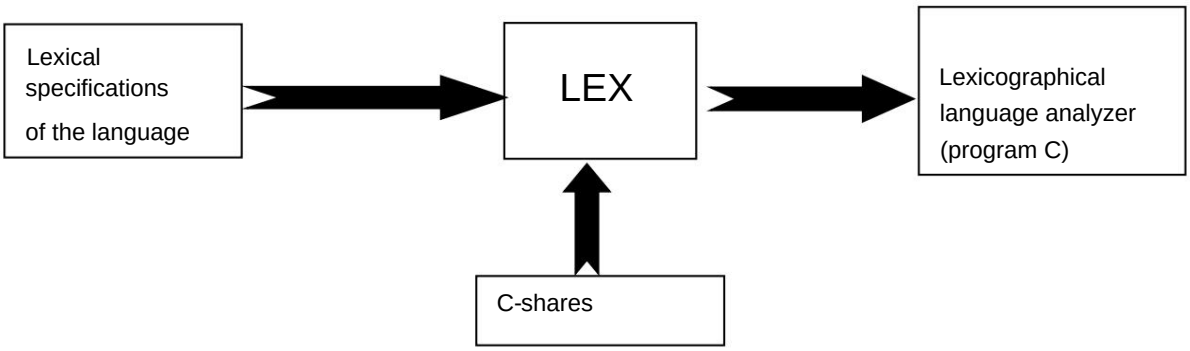


Fig.1 Lexical analyzer generator.

The lexicographic analyzer must then be compiled before it can be used.

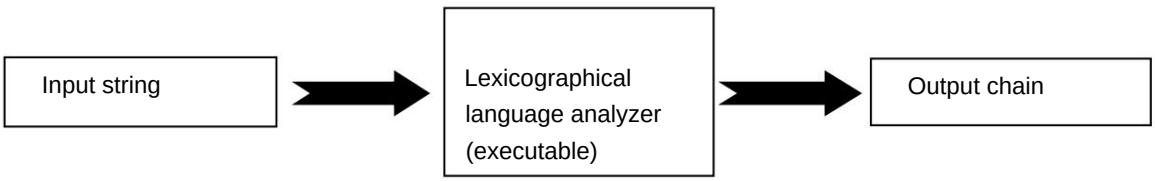


Fig.2 Using the lexical analyzer.

The data for this executable program must be a string conforming to the lexical specifications. As the input stream is processed, actions are executed each time a token is recognized. If the input stream contains expressions that are not recognized, they are copied to the output without any processing.

Machine Translated by Google

Noticed:

The program generated by Lex is stored in a file named **lex.yy.c** which contains The **yylex() function.** This function recognizes defined expressions and performs the processing. asked about these.

The lexical specifications of the language being processed are described in a language that will be presented in section 2.2. Before that, we will give two examples. The sole purpose is to to give an idea of this language.

Examples:

1. A program that removes spaces and tabs at the end of each line

The specifications are as follows:

```
%%
[ \t]+$ ; (note: there is a space before \t)
```

The specification contains **the delimiter %% to indicate the start of the specification rules.** In this example there is only one rule which says that every time we encounter one or more blanks or tabs , No action is performed. The square brackets define The class formed by the whitespace and tab characters; the + sign means "one or more" characters of this type (repetition); finally, $ indicates "end of line". No action is if specified, then the yylex() function will ignore that configuration. Any other combination will to be copied on output.

2. Program that replaces several whites or tabs with a single white.

```
%%
[ \t]+ printf(" ");
```

Here we see the use of an action (printf).

Machine Translated by Google

2.2 LEX Sources

The general format of a lexical specification for a language in LEX is as follows:

definitions

%%

rules

%%

user subroutines

The definitions, subroutines, and the second %% (if there are no subroutines) programs) are optional.

The definitions will be addressed in section 2.6.

The first %% must always be present to mark the beginning of the period. The most A simple LEX program is as follows:

%%

No definition, no rules. The effect of this program is to copy the string from input to output without any modification.

In normal mode, the rules have 2 parts:

• in the first part **ÿ the expression** that must be found;

• in the second part **ÿ the action** to be taken when this expression is found.

~~Examples:~~

1. **%%**

int printf("found keyword INT");

This program searches the input data for the word "int" and displays the output. message "found keyword INT".

2. **%%**

hello printf ("hello");

world printf("world");

sea printf ("sea");

This program can represent the beginning of an English-to-French translator.

Machine Translated by Google

2.3 Typed words (tokens)

In the preceding examples, the expressions are reduced to a token.
A token is a string of characters (letters or numbers) that we want to detect. In the case
In general, an expression contains token characters and operators that specify
repetitions, choices or other rules for composing these tokens.

Example:

[^abc]+ defines the repetition (+) of letters of the alphabet except a, b and c(^).

We will now go into detail about each operator.

2.3.1 Operators

The characters used as operators are as follows:

" \ [ ] ^ - ? · * + | ( ) $ / { } % < >

If you wish to use them as characters in a token, they must be preceded by the symbol \.
The action of these operators is as follows:

• The operator **"** indicates that everything between the signs " " should be considered as text of a token;

• The **** operator indicates that the sign that follows it should be treated as a character of a token.
This same operator can be used for the portable specification of special characters as :

\n	**newline**
TAB	
\b	**backspace**

Example: Expressions

xyz"++"

"xyz++"

xyz\+\+

They all have the same meaning. These expressions search for the string xyz++ in the stream
The choice between these two options is a matter of personal preference.

Machine Translated by Google

Noticed

If we had written xyz"+ "+, we would have expressed the repetition of the token xyz+.

• The operator **[ ]** delimits classes of characters. The meaning of operators is not taken into account when these are between [ ], with the exception of the operators \, **-** and **^** ;

Example : **[abc]** is a token consisting of a single character which can be either a, b, or c.

• the operator **-** indicates the range of variation;

Example [a-z0-9<>] means the class of lowercase characters between a and z, at to which we add the numbers between 0 and 9, the symbols "smaller" and "larger".

• The **^** operator indicates that the class is composed of the complement of the characters specified. This operator always appears between square brackets.

Example: **[^abc]** represents the class of all characters except a, b and c;

[^a-zA-Z] represents the class of characters that are not letters.

Noticed :

The character ^ must be in the first position between the brackets.

• the operator. means any character;

• The operator ? means that the element in front of it is optional;

Example: **ab ?c** means either the string **abc** or **ac,** that is, the appearance of b in this This expression is optional.

• Repetition operators

• The ***** operator indicates any number of occurrences (including 0) of the character or the character class that precedes it;

• The **+** operator signifies one or more occurrences of the character or class of previous characteristics;

Example :

[az]+ means all lowercase strings;

[A-Za-z][A-Za-z0-9]* all strings that begin with a letter. This expression is usually used to recognize identifiers in languages of

Machine Translated by Google

programming.

• the **|** operator is used in the construction of alternative expressions;

Example: **ab|cd** means either ab or cd.

• The operator **( )** is used to group alternative expressions into larger expressions complicated;

Example: **(ab|cd+) ?(ef)*** can represent "abefef", "efefef", "cdef", "cddd" , but not “abcd”, “abc” or “abcdef”.

• The **^** operator is used to recognize expressions that appear at the beginning of a line. This usage should not be confused with the complementarity operator, because This always appears between brackets, as specified above;

• The **$** operator is used to recognize expressions that appear at the end of a line ;

Example: **ab$** recognizes the expression ab if it appears at the end of the line

ab/\n is equivalent to ab$

• The operators **{ and }** specify either repetitions, if numbers are between them, or a definition if there is a name between them;

Example: {digit} searches for the string "digit" and replaces it with the corresponding definition;

a{1,5} searches for occurrences of groups of 1 to 5 "a";

a{2, } searches for occurrences of "a" 2 or more times;

a{3} searches for exactly 3 occurrences of "a"

2.3.2 Operator Priority

Operators placed on the front lines have a higher priority than those placed on The last lines (decreasing priority). On the same line, operators have priority. next decreasing order:

Machine Translated by Google

Escape characters	\special characters
Expressions in brackets	[…]
Expressions in quotation marks	"…"
Operator used for grouping	(…)
Definitions	{name}
A character, repetition	* ? +
Concatenation	
Context	$ ^
Alternative	|

Note: LEX accepts expressions with a maximum length of 100 characters.

2.3.3 Actions

When an expression (defined as described above) is recognized, the program

The function generated by LEX will perform the actions specified following the definition. The actions are,

Generally, fragments of code written in C language, provided by the user.

The simplest action is the null action ";", that is, we recognize the expression and we

It does nothing.

For more complex actions, there is a series of functions and global variables.

as:

• the global variable **yytext,** which is a pointer to a string of characters that contains

the expression found;

Example: To print the expression found, the action to take is:

[az]+ printf("%s", yytext) ;

This action is very commonly used; it can be written more simply:

[az]+ ECHO;

• the global variable **yyleng** which represents the number of characters in the expression found;

Example: To count the expressions found and the number of characters processed, we write:

[a-zA-Z]+ {words++; chars +=yyleng;}

The last character of an expression can be accessed as follows:

yytext[yyleng-1]

Machine Translated by Google

Concatenation

To concatenate two expressions, we use the `yymore() ` function . indicates that the next recognized token will be added to the current value of **yytext** instead to replace this value. The value of **yyleng** is modified accordingly.

Example: "xyz+" {printf("%d **ÿ** %s\n", yyleng, yytext);

yymore();}

printf("%s", yytext);

As soon as we detect xyz+ in the input stream, we will display 4 **ÿ** xyz+ and the expression that the following will be displayed xyz+"expression".

Entrance: xyz+32

Exit: 4 ÿxyz+

xyz+32

If you want to remove the last characters from an expression, you use the function **yyless(n)** where n represents the number of characters we want to keep in **yytext.**

Examples: 1. "integer" { printf("%s\t ", yytext);

yyless(2); }

printf("%d **ÿ** %s\t ", yyleng, yytext);

As soon as we detect "integer" in the input stream, we will display "integer", we keep in **yytext** the first 2 characters and from the 3rd character onwards the analysis is restarted according to the lexical rules are defined. The output of this example is therefore as follows:

Input: integer 32

Output: integer **1ÿt 1ÿe 1ÿg 1ÿe 1ÿr 1ÿ"** " **1ÿ3 1ÿ2**

2. char buffer[400];

%%

\n.* { strcpy(buffer, yytext+1);

yyless(1);

}

This specification saves the current line in a local buffer. The expression \n.* detects a newline character and the entire line following it. After that, the line's contents are saved in the buffer , It was returned to the lexical analyzer for analysis.

Machine Translated by Google

2.4 Access to input/output subroutines

For this, we use the following functions:

- **input() ÿ** gives the following character;
- **output(c) ÿ** writes the character c to the output;
- **unput(c) ÿ** puts the character c into the input data stream to be read later by input().

Another function, which can be redefined by the user, is **yywrap(),** a function that is called every time EOF is detected. It can be used to open a new file at the end of the current file to continue lexical analysis. If it is not modified, **yywrap()** returns 1 when EOF is detected.

2.5 The ambiguity of the rules

LEX can work with specifications that contain ambiguities. When The input expression corresponds to several rules; LEX chooses according to the following criteria:

1. The longer expression is preferred.
2. For the same length, LEX prefers the rule that appears first.

<u>Example:</u> integer printf("Keyword\n");
[az]+ printf("Identifier\n");

If the input data stream contains "integer", it will be treated as a word. key, because even if both rules detect expressions of the same length, the integer rule is defined first.

In LEX, a character is only taken into account once.

<u>Example:</u> she s++;
Hey h++;
\n |
. ;

We'll ignore everything except "she" and "he". Since "she" includes "he", LEX will generate a a program that will not recognize instances of "he" within "she". If one desires

Machine Translated by Google

To recognize instances of "he" within "she", the REJECT action must be used.
means "move to the next alternative". Generally, REJECT is useful when you're not looking for...
the division of input streams into terminals, but we want to detect the appearance of certain
configurations that are partially overlapping.

2.6 Definitions

The user needs new facilities to define the variables that will be
used by LEX. These can appear both in the definitions section and in the
rules section. LEX transforms the rules into a program. Any action that is not
The code performed by Lex is copied into the main program. There are two rules for achieving this:

1. Any line that is not part of a rule or action and that begins with a blank.
or a TAB is copied into the program. Variables defined before the first appearance
The elements of the expression %% are global variables. The comment lines are also copied.
in the source code.

2. Any string of characters enclosed between %{ and %} is copied as is (the delimiters are
eliminated). Any line not included between these delimiters and starting at the first
position will be treated by LEX as a substitution with the format:

name **expression**

and determines the association of the expression with the noun. The two fields must be separated.
less by a space or a TAB and the name must begin with a letter.
Example: recognizing floating-point numbers

```
D           [0-9]
E           [Ee][-+]?{D}+
%%
{D}+                        |
{D}+{E}                     printf("integer\n");
{D}*"."{D}*({E})?           printf("real\n");
```

Machine Translated by Google

2.7 Usage

Compiling a LEX program is done by following the steps below:

1. In the functions section (last part) of your lexical analyzer, you need to add:

```
%%
int yywrap()
{
return 1;
}
int main()
{
 yylex() ;
 return 1;
}
```

2. Generating the program in C language using LEX

lex source.lex or flex source.

This operation will generate the C program **lex.yy.c**

3. Compiling programs with a C compiler.

gcc lex.yy.c -o an-lex.exe

will generate the executable lexical analyzer an_lex.exe

2.8 Lex and YACC

LEX provides a function yylex() - a lexical analyzer. yylex() is exactly the name requested by YACC for the lexical analyzer. This function is called by YACC for recognize the tokens in the input stream. In this case, the LEX rule must end with

return token_name;

so that YACC is informed of the token found by the lexical analyzer.

The main() function must also be removed from the lexical analysis file in LEX.

Machine Translated by Google

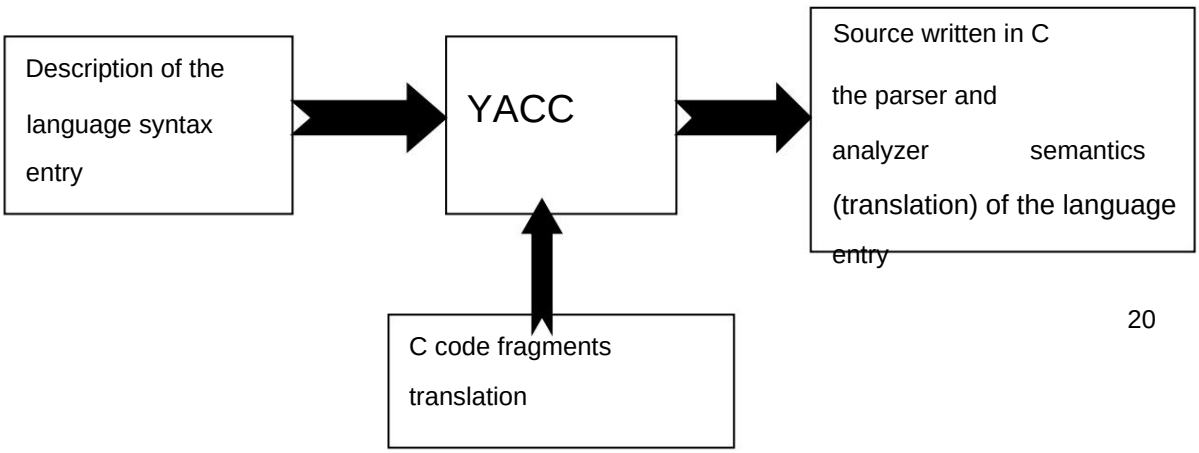
3. YACC: Syntax analyzer generator

YACC - Yet Another Compiler Compiler

The input data stream used by different programs has a structure well defined. In fact, any program can be seen as a parser that accepts a certain "input language". An "input language" can be as complicated as a language programming or as simple as a string of numbers.

YACC is an instrument that, starting from the description of the syntax of the "language" The input" generates the associated parser. Users specify the structure of the input language and C code fragments that must be executed each time a The structure is recognized. If the "input language" is a computer language, the C code performs generally a translation of the structure of the source (input) language into instructions execution machine assembler.

YACC transforms the input language specifications into a program written in C language which performs syntactic analysis of input data.



Machine Translated by Google

Fig.1 YACC

Using YACC, you can create compilers for C, Pascal, Fortran, or... compilers for programming languages for graphics printers, etc.

3.1 Introduction

YACC users provide input stream specifications, which include rules describing the syntactic structure, program fragments that represent the actions to be performed and specifications for the primary processing of input data. YACC generates a function that performs input data validation. This function calls the function of the lexical analyzer to recognize terminals. The conformity of the sequence of terminals with the input language syntax are then checked. Each time a rule is Once recognized, the instructions specified by the user are executed (fragments of C code). translation).

The syntax rules are described by defining for each non-terminal (placed at left of :), the composition of terminals and non-terminals (placed to the right of :).

Example:

Data: day '.' month '.' year;

Data: day '/' month '/' year;

where "day", "month", "year" are previously defined structures, terminals (tokens) already recognized by the lexical analyzer which passes them to the syntactic analyzer.

The symbols ":" and ";" are delimiters for a syntax rule.

We will then detail how to express rules and actions.

Machine Translated by Google

3.2 Basic Specifications

The format of a YACC specification is as follows:

Statements

%%

Rules

%%

Programs

The declarations section may be missing, as well as the program section. The gaps Tabs are ignored. Comments are limited by /* …. */ as in C. The rules have the following format:

A: BODY;

"A" represents a non-terminal symbol, and "BODY" represents a string of names and Letters. The symbols ":" and ";" are delimiters. Names can be symbols terminal or non-terminal. YACC requires that the names of terminal symbols be declared as such in the declarations section.

Example:

%token name1 name2 …

Any undefined name in the declarative part is considered a non-symbol. terminal. Any non-terminal symbol must appear in the left-hand side of a rule at least once.

Names can be of arbitrary length and are formed by letters, ".", "_", and numbers (not at the beginning of the name). Uppercase and lowercase letters are distinct.

A letter is specified by a character between ' and '. As in C the character "\" is Used for the portable specification of special characters (newline, return, tab, etc.). For For technical reasons, the NULL character (\0 or 0) should not be used in the rules. syntactic.

If several rules have the same left-hand side, you can use the character "|". It is not It is not necessary for all rules with the same left-hand side to appear together. All This sometimes makes it easier to understand the specifications.

Example:

A: BCD; ⟷ A: BCD

Machine Translated by Google

```
A: EF;          | EF
A: G;           | G
                ;
```

If a non-terminal symbol needs to recognize an empty string, we will write:

```
Empty :         ;
```

Among all the non-terminal symbols, there is one very important one. This is the **Start symbol.** The parser recognizes this symbol. By default, the symbol of beginning (axiom) is considered to be the **first name** on the left side of the **first rule.** If desired, this explicit symbol can be declared using the keyword: **%start**

Example: %start symbol

The end of the input stream is signaled with a special symbol called an "end marker". If The symbols found up to the end marker correspond to a syntactic rule; the parser has It has finished its activity. Otherwise, it reports an error.

3.3 Actions

Each syntactic rule is associated with actions. These actions can return different values and can use the values returned by other actions. An action is a fragment of C code, placed between curly braces.

Example: Syntax rule with actions

```
A: '(' B ')'
        {printf("message %d\n", i);
         i++}
```

To facilitate communication between actions and the parser, we use the **The $ symbol is represented** as follows:

- To return a value, the action must assign a value to the pseudo-variable $$.

  Example: an action that simply returns the value 1

  ```
  { $$=1; }
  ```

- To obtain the returned value of a previous action, the current action must use the pseudo-variables $1, $2, … which refer to the returned values of the components of the right side of a ruler, traversed from left to right.

  Example: 1. Consider the rule

Machine Translated by Google

A: BCD;

then $2 represents the value returned by C and $3 the value returned by D.

2. expr : '(' expr ')' { $$ = $2; }

The value returned by this action is the value of "expr" which is between the parentheses.

By default, the value of a rule is the value of the first element of that rule (that is-
(say $1). For this, the rules of type

A : B; ⟷ A: B

{ $$ = $1;}

require no action.

In the previous examples, the actions were placed at the end of the rule. Sometimes, it
Actions must be taken before a rule is fully analyzed. YACC
allows you to write actions inside the rule.

Example :

HAS : B

{ $$ = 1;}

C

{ x = $2; y = $3;}

The effect is to assign the value 1 to x and to assign the value returned by C to y.

Actions that are not at the end of a rule are considered by YACC as
new non-terminal symbols; the rule that analyzes these symbols is the empty string.

In some applications (e.g., C compilers) the output is not the direct result
actions. Actions build a grammatical parsing tree structure that generates the
output. These syntax trees are constructed using the following technique:

1. We write a function (in C) called "node" such that the call

node(L, n1, n2)

generates a node with the label L, having descendants n1 and n2, and which returns the index of
node created.

2. The analysis tree can be generated using actions of the following type:

expr: expr '+' expr

{$$ = node('+', $1, $3); }

Machine Translated by Google

The user can define other variables that can be used by actions. These variables must be defined in the declarative part of a YACC specification between the symbols **"%{ "** and **"%}".**

Example:

```
%{ int toto = 0; %}
```

The variables used by YACC have names that begin with "yy" and the user must avoid defining variables with such a name.

3.4 Lexical Analysis

The lexical analyzer is a function that must be named **yylex().** The function must return an integer that represents the " **token number read"** in the input stream. If there is a The value associated with this token will be assigned to the external variable **yylval.**

The lexical analyzer and the syntactic analyzer must "agree" on the numbers of each token. These numbers can be chosen by the user or by YACC. In both cases The C "#define" mechanism is used. By default, YACC chooses these numbers. For a The YACC character selects the numeric value for that character and numbers for the others. greater than 257. If the user chooses the numbering, they must **always** choose for these tokens of numbers > 257.

To assign a number to a token, its first appearance in the declarations region must be followed by a positive integer. Tokens and characters that are not followed numbers of an integer are assigned default numbers, as described above.

Example :

```
DIGIT - is declared in the declarations section
yylex(){
        extern int yylval;
        int c;
        c = getchar();
        switch(c){
                case '0':
                case '1':
                ...
```

Machine Translated by Google

```
                    case '9':
                        yylval = c - '0';
                        return (DIGIT);
                }
        }
```

This example will return the number corresponding to DIGIT and its value each time we finds a number in the input stream.

3.5 Ambiguity and conflicts

A set of syntactic rules is considered ambiguous if the same The input string can be structured in several different ways.

Example :

expr : expr '-' expr;

is a natural way to describe how a mathematical expression is formed.

But this rule doesn't tell us how to interpret an expression like:

expr – expr – expr

It can be structured as follows:

(expr – expr) – expr

or like:

expr – (expr – expr)

YACC detects this type of ambiguity. A more detailed report can be obtained using The –v option is used when compiling the YACC source file. You need to look at the y.output file.

To get an idea of how conflicts of this type are resolved, one must take a look inside the parser. The parser is a Battery-powered automaton that can perform four functions: **movement, reduction, acceptance** and **error.**

1. The **move** function adds a new symbol to the parsed string. This function fills the automaton's battery.

Machine Translated by Google

2. The **reduction** function is the action performed by the automaton when it has recognized a Rule. This function empties the automaton's battery.

3. The **acceptance** function is a function that indicates that the input stream has been analyzed. and that it meets the specifications.

4. The **error** function indicates that, in the input stream, there exists a substring that does not corresponds to no specification.

Let's return to the previous example. We see that after reading the string "expr - expr" We have a conflict of the **displacement/reduction type.** In some cases, a conflict of the **reduction** type. **/ reduction** may appear, but never a **move/move** type conflict .

The rules for eliminating these conflicts are as follows:

1. In the case of a **displacement/reduction** conflict , the displacement wins.
2. In the case of a **reduction/reduction** conflict , the rule that appears is chosen. first in the specifications.

So our input string will be grouped: **expr – (expr – expr)**

Example: Here is another example of the power of YACC to resolve ambiguities. Let's consider an example of a programming language that allows "if" type structures – then – else".

```
statement   :   IF '(' cond ')' statement
            |   IF '(' cond ')' statement ELSE statement
```

In these IF and ELSE rules, IF and ELSE are terminals, and "cond" and "statement" are non-terminators. terminals. “cond” describes a condition and “statement” describes a chain of actions.

If the input is of type:

```
IF (C1) IF(C2) S1 ELSE S2
```

It can be structured in 2 modes:

```
IF (C1) {
        IF (C2) S1
         }
ELSE S2
```

Or

```
IF (C1) {
        IF (C2) S1
        ELSE S2
```

Machine Translated by Google

}

The second interpretation is the usual one for many programming languages.

Let's analyze, for this example, the situation where the parser has managed to analyze only :

IF (C1) IF (C2) S1

This string can be considered an instance of the rule

IF (C1) statement

After reading the rest of the expression

ELSE S2

One might think that this chain will be considered an instance of the rule

IF (C1) statement ELSE S2

This was true if the expression was reduced before making the move. But as the Disputes involving displacement/reduction always result in the displacement party winning, ELSE will be moved and, after reading the S2 symbol, the reduction of the IF – ELSE rule of the part The right will be made, followed by the reduction of the rule

IF (C1) statement.

SO , We obtain the second interpretation, the correct one.

To avoid the risk of problems related to ambiguities, it is best to... avoid when giving your syntax rules (see the course).

3.6 Priorities

There are situations where the rules defined previously are not sufficient for specifying the notion of priority. This is, for example, the case in the syntactic analysis of expressions arithmetic. The majority of arithmetic constructions can be described using the concept of operator priority levels and information about their associativity.

The association and priority of operators can be specified in the area declarative. This is done using the keywords:

%left

%right

%noassoc

followed by a list of terminal symbols (tokens). All the symbols that are found on a The same line has the same priority. Terminal symbols that are on the first

Machine Translated by Google

lines have a lower priority than those on the following lines (priority increasing).

Example :

```
%left '+' '-'
%left '*' '/'
```

These lines describe the priority and associativity of four arithmetic operators. + and - have the same priority and left associativity due to the term "left"; * and / have a higher priority greater than the previous operators and left associativity.

The keyword *%right* is used (as you've already guessed) for associativity to right and the keyword *%noassoc* is used to describe operators without associativity (like some FORTRAN operators).

Example: Specifications for arithmetic expressions

```
%right '='
%left '+' '-'
%left '*' '/'

%%
expr:     expr '=' expr
        | expr '+' expr
        | expr '-' expr
        | expr '*' expr
        | expr '/' expr
        | NAME
        ;
```

The input string conforms to this specification:

```
a = b = c*d – e – f*g
```

will be structured as follows:

```
a =(b= (((c*d) –e) –(f*g) ))
```

When using such a mechanism, it is also necessary to specify a priority for the operators. unaires (like ' - (' before a number). The keyword **%prec** modifies the priority of an operator within the framework of a syntactic rule. It must appear immediately after the rule has been defined. In the following example, the unary operator ' is given - 'the same priority as that of the operator '*'.

Example :

Machine Translated by Google

```
%right '='
%left '+' '-'
%left '*' '/'

%%
expr:       expr '=' expr
        |   expr '+' expr
        |   expr '-' expr
        |   expr '*' expr
        |   expr '/' expr
        |   '–' expr            %prec '*'
        | NAME
        ;
```

A terminal symbol declared using **%left, %right, %noassoc** is no longer necessary (but it can) be declared using **%token.**

3.7 Error Handling

Error handling is a difficult field. Often, stopping is unacceptable. Data processing is initiated if an error is found. Therefore, the recovery problem must be solved. analysis after an error is encountered. The general algorithm for resuming analysis involves the elimination of certain terminals in the input string, in order to resume the analysis in front of a correct expression.

To enable error control, YACC offers a simple, but sufficient, method. In general, the terminal symbol **"error"** is reserved for handling syntax errors. This token can be used in syntax rules and for actions to correct them. The parser empties its stack until it reaches the state in which the "error" token is allowed; then he performs the corresponding action.

To avoid generating too many error messages, the parser, after the Error detected, remains in error state until 3 successive tokens have been moved and reduced correctly. If, during this time, a new error is detected, it is not generated a new error message.

<u>Example :</u>

Machine Translated by Google

1. **statement : error;**

This means that in the case of a syntax error, the parser will ignore the expression. where the error occurred. More specifically, the parser will continue reading the input symbols by searching for 3 consecutive tokens that are allowed for the expression current and will begin the analysis with these.

2. statement : error ' ;'
;

indicates that in the case of a syntax error, the parser will ignore all the Terminals and non-terminals up to ';'. All symbols up to ';' will not be They will be moved and ignored. When the symbol ';' is encountered, the rule will be reduced.

Another form of error handling can be used in programs interactive. We want the line that had errors to be reinstated once it has been displayed the error message.

Example :

```
input:      error '\n'      {printf("Enter the last line");}
            input
                            {$$ = $4 ; }
        ;
```

3.8 Usage

Compiling a YACC program involves two steps:

1. Generating the program in C language using YACC:

YACC source_filename

This operation will generate the file **y.tab.c,** which contains the function **yyparse().** When this The function is called, and it will in turn call the **yylex()** function to obtain the symbols. input. If an error is reported, yyparse() returns 1.

Machine Translated by Google

If we want to generate the file y.tab.h which contains the #define type declarations which associates the user-defined token names with the YACC code for each token.
use the -d option when compiling .

YACC -d source_filename

The user must specify in the YACC source file, in the sub- area programs, other functions. For example, the main() function which calls the function yyparse(). A yyerror() function must also be defined (a function defined in the library of YACC) which indicates that a syntax error has been encountered.

Example :

```
hand()
        {
        return yyparse();
        }
```

And

```
yyerror(char *s)
        {
        fprintf(stderr, "%s\n", s);
        }
```

• Compiling this program:

gcc y.tab.c

For compiling the lexical analyzer files (lex.yy.c) and the analyzer syntax (y.tab.c) requires using the following command:

gcc y.tab.c lex.yy.c –ll –o executable_name.exe

Be careful to maintain this order for the gcc parameters .

Recommendations for writing specifications

There are some conventions that facilitate understanding, although they are not obligations:

1. Use names that begin with ' t ' or capital letters for tokens.
2. Put the syntax rules and actions on different lines.
3. Group all the rules with the same left-hand side together.
4. Place the sign ' ; ' from the end of a rule on a separate line.

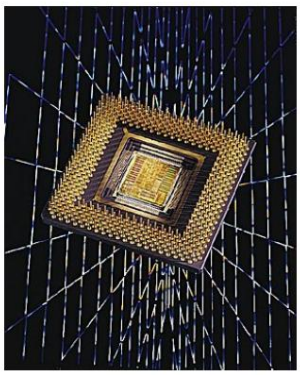
Machine Translated by Google

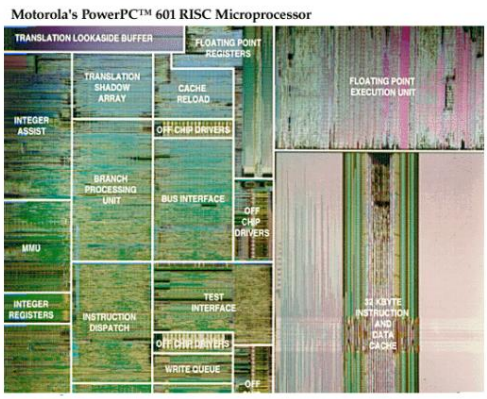
5. Position the actions of a rule with an offset of 3 tabs and the syntax rules with a shift of 2 TABs.

All these recommendations make it easier to read the specifications file. syntactic.

Design of a RISC-type microprocessor with pipeline

Machine Translated by Google





Note: The Material Architecture course and practical work in the 3rd year of MIC are a **mandatory prerequisite** for this practical work.

1. Objective

2. Part One - You will design and implement a microprocessor in VHDL with **a pipeline** corresponding to the **assembler instructions : addition, subtraction, multiplication, division, copy and assignment (without the jump and assignment instructions) comparison).** You will then synthesize and optimize this microprocessor from the point of view of the operating frequency and you will then implement it on the Xilinx FPGA.
This section is mandatory.

For the 2019-2020 academic year, FPGA implementation is optional due to Remote work. Synthesis and performance analysis of the processor obtained (operating frequency, consumption) remain mandatory.

Machine Translated by Google

3. Second part - you will add the units to your microprocessor architectural requirements for implementing jump and comparison instructions (by (Example: the branching unit). You will then synthesize and optimize this microprocessor from the point of view of operating frequency and then you will implement it on the Xilinx FPGA. **This second part is optional in 2019-2020.**

To guide you in the design of this microprocessor, we will, in the second In this section of the document, we will present the register-oriented instruction set that you will use. In the third section, we discuss the architecture of the microprocessor that we we propose to implement.

2. Assembly language

The register-oriented instruction set you will use is shown in Table 2. *Ri, Rj* and *Rk* represent the register numbers requested by the instruction. *@i* and *@j* represent Memory addresses. The instruction format is fixed size. The value _ is used as padding , in instructions requiring fewer than 3 operands. In other words, All instructions occupy 4 bytes, as illustrated in Table 3. The games not all commercial microprocessors possess this instruction set. characteristic. We propose that you design a set of register-oriented instructions with a fixed size format, to facilitate your design.

Operation	Code	Instruction format				Description
		OP	AB		C	
Addition	0x01 ADD Ri			Rj Rk	[Ri] ÿ [Rj] + [Rk]	
Multiplication	0x02 MUL Ri			Rj Rk	[Ri] ÿ [Rj] * [Rk]	
Subtraction	0x03 SOU Ri			Rj Rk	[Ri] ÿ [Rj] - [Rk]	
Division	0x04 DIV		Ri	Rj Rk	[Ri] ÿ [Rj] / [Rk]	
Copy	0x05 COP Ri			RJ	_	[Ri] ÿ [Rj]
Assignment	0x06 AFC Ri			j	_	[Ri] ÿ j
Loading	0x07 LOAD Ri @j _					[Ri] ÿ [@j]
Backup	0x08 STORE @i Rj [@i] ÿ [Rj]				_	

Table 2. Instruction Set

Machine Translated by Google

Instruction: ADD		R1	R9	R4
Hexadecimal: 0x01		0x01	0x09	0x04
Binary:	00000001 00	000001 00001	001 00000100	

Table 3. Example of an instruction

Compared to the memory-oriented instruction set of the Automata and Languages practicals, two New instructions have appeared: *Loading* and *Saving.* The instruction to loading allows copying the contents of memory at address @j into the *Ri* register . The save instruction allows the contents of the file to be copied into memory at address *@i. RJ* register . These are the only instructions allowing communication with memory. Such a The RISC processor is described as a RISC **Load/Store** processor . Note that the instruction set used in Automata and Languages practical work is not suitable for this type of microprocessor.

3. RISC Microprocessor Architecture

We propose to implement a **RISC** -type architecture here with a **5-pipeline stages** (Figure 4). This architecture must operate on **8 bits.** We reproduce the following in the appendix. **Michel Cubero's 3rd-** year Computer Structure and Functioning course **Castan** regarding the instruction cycle, the pipeline and contingencies.

Our microprocessor will have:

- An arithmetic and logic unit, *section 3.1 ;*
- A dual-port read register bank, *section 3.2 ;*
- An instruction memory, *section 3.3 ;*
- A data memory, *section 3.3 ;*
- A data path, *section 3.4;*
- A control unit, *section 3.5 ;*
- A hazard detection unit, *section 3.6 ;*
- Pipeline architecture on 5 floors.

Machine Translated by Google

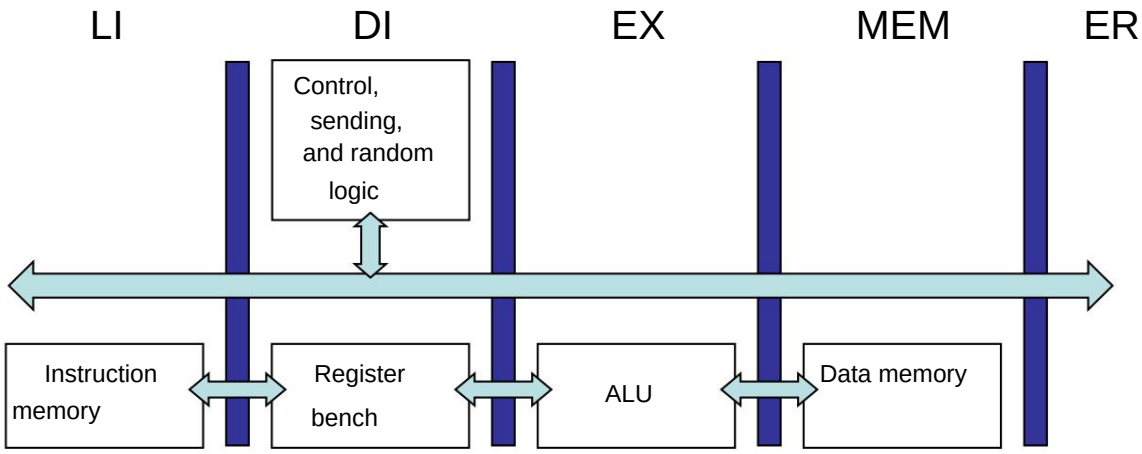


Figure 4. RISC microprocessor architecture with 5 pipeline levels

3.1 The arithmetic and logical unit

We propose to implement an ALU with the interface shown in Figure 5. The *Ctrl_Alu* signal will inform regarding the operation to be performed: Addition, Subtraction, Multiplication and Division. The flags : N , W, *Z* and *C* represent, respectively, a value negative output *(S* < 0), an **overflow :** size of *A* OP *B* > 8 bits), a zero output *(S* = 0) and the Carrying **(or carrying)** in addition. Arithmetic operations **can** be performed using the corresponding functions ('+', '-', '*', '/'), **provided in the Xilinx library.** The FPGAs used (Spartan 6) contain hardwired multipliers, so you can implement the multiplication function using simply the * sign in VHDL .

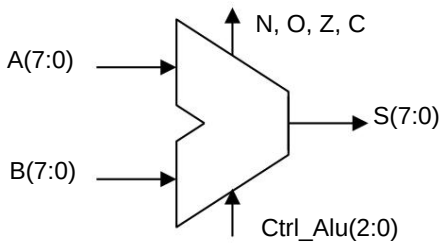


Figure 5. UAL

We are not asking you to implement the division in the 2019-2020 academic year.

As a reminder, we show you an example of how a multiplication works on binary numbers (on 4 bits), in figure 6.

Machine Translated by Google

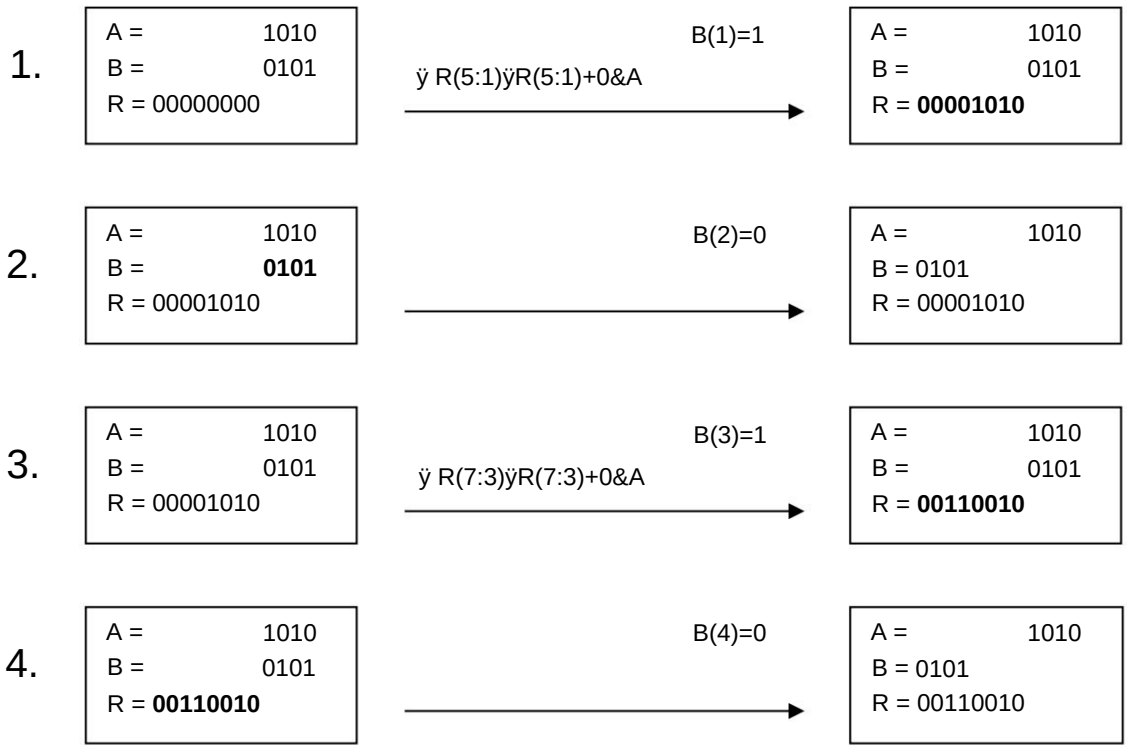


*Figure 6. Flow of a 4-bit multiplication: 10 * 5 = 50*

3.2 Dual-port read register bank

We propose to build a bench of 16 8-bit registers with dual access read and write access. The The diagram of this register is presented at the Figure 7. The *RST* reset signal is active at 0: the contents of the register bank are then initialized to *0x00. @A* and *@B* allow reading two registers simultaneously. THE values

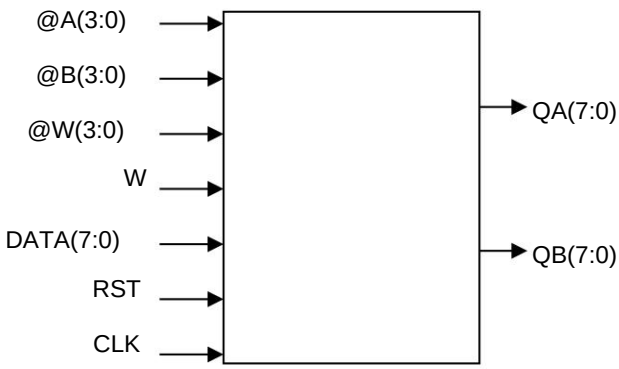


Figure 7. Register bench

The corresponding data is propagated to the *QA* and *QB* outputs . Writing data to a Register access is done via the *@W, W* , and *DATA entries. W* specifies whether a write operation should be performed. Completed. This entry is active at 1, for a write operation. When the write operation is active, the data The data present on the *DATA* input are copied into the address register *@W.* It is assumed that the The reset and writing will be done synchronously with the clock.

Machine Translated by Google

Bypass D ÿ Q

Sometimes, during execution, the processor simultaneously makes a read request and writing to the same register. This constitutes a data hazard. This hazard can be handled by the sending unit. However, in order to simplify the design of the sending unit, we propose to implement this functionality directly in the registry:

If writing and reading are on the same register, then the output QX ÿ *DATA.*

3.3 Memory Bench

Our architecture contains two memories: one memory for data and one memory for the instructions. Their structure is shown in figure 8.

The data storage must allow read and write access. The address of the area Memory is provided by the @ input. To perform a read operation, *RW* must be set to 1 and To perform a write operation, *RW* must be set to 0. In the case of a write operation, the content of The *IN* input is copied to memory at address @. The reset, *RST,* will initialize the Memory contents at *0x00.* Reading, writing, and resetting will be synchronous with the *CLK clock.*

To simplify the design, we propose an instruction memory structure simpler. We will assume that the program to be executed by the microprocessor is already stored in this memory. Furthermore, at runtime, we prevent any modification of the the contents of this memory. It is similar to a **ROM.** It is therefore devoid of inputs *RST, IN* and *RW.* As with data memory, reading will be synchronous with the CLK clock .

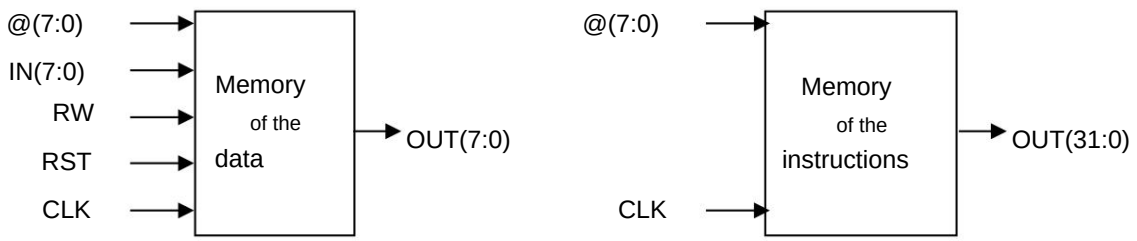


Figure 8. Data memory and instruction memory

Machine Translated by Google

3.4 Data Path

We will now begin integrating the various elements previously developed to design a first version of the processor. We will create a path data (shown in illustration 6). To guide you in creating this path of Given this, we advise you to proceed step by step. In other words, we advise you to begin by creating a data path that only takes into account one of the instructions. You can then test your implementation to correct any errors. possible. Then, you can integrate a new instruction. You will thus loop through so many that there are still instructions to implement. The rest of this section presents an example approach.

1 - We begin by implementing the *AFC instruction.* This instruction allows us to copy a constant in a destination register. Knowing that modifying a register should not be performed only at floor 5, the identifier of the destination register and the value to be assigned to it must be propagated to this floor. The path followed by this instruction is therefore the following :

1.1. At the first floor level, the various fields constituting the instruction are placed in the first pipeline *(LI/DI) ;*

1.2. At the second stage, no value selection in the register bank is Required: the current operation code, the value to copy, and the destination register are simply propagated in the pipeline *(DI/EX) ;*

1.3. At the third floor level, no arithmetic execution is necessary for the copy: the current operation code, the value to copy, and the destination register are simply propagated in the pipeline *(EX/Mem) ;*

1.4. At the fourth floor level, no memory modification is necessary For copying: the current operation code, the value to copy, and the register destination are simply propagated in the pipeline *(Mem/RE) ;*

1.5. On the fifth floor, we have all the necessary information To complete the copy: the value to copy and the destination register. This modification This is done using the write input of the register bank.

Machine Translated by Google

The corresponding diagram is given in illustration 1. In this illustration, *OP* represents the operation code and A, *B ,* and *C* the operands. Note, in passing, that the path taken by This operation requires, at level 5, the use of a component located schematically at the level of the 2nd floor. But, in no way, does the path in question pass through a same pipeline.

2 - Based on this diagram, the *COP* instruction can be added. The difference compared to the The previous scheme is the obligation, when moving to level 2, to propagate the value associated with the source register and not the identifier of that register. To make these changes without... To avoid disrupting the proper functioning of the *AFC instruction,* we use a multiplexer. depending on the operation code, it propagates parameter *B* or the value contained in the register pointed to by parameter B. The resulting diagram is given in illustration 2.

3 - Note that the previous illustrations include an arithmetic and logical unit without to use them. We will therefore include the arithmetic instructions to connect this ALU to the data path. Using multiplexers again to determine the elements To propagate, we obtain illustration 3.

4 - By proceeding in the same way for the implementation of the *LOAD* and *STORE instructions,* we obtain illustrations 4 and 5.

Machine Translated by Google

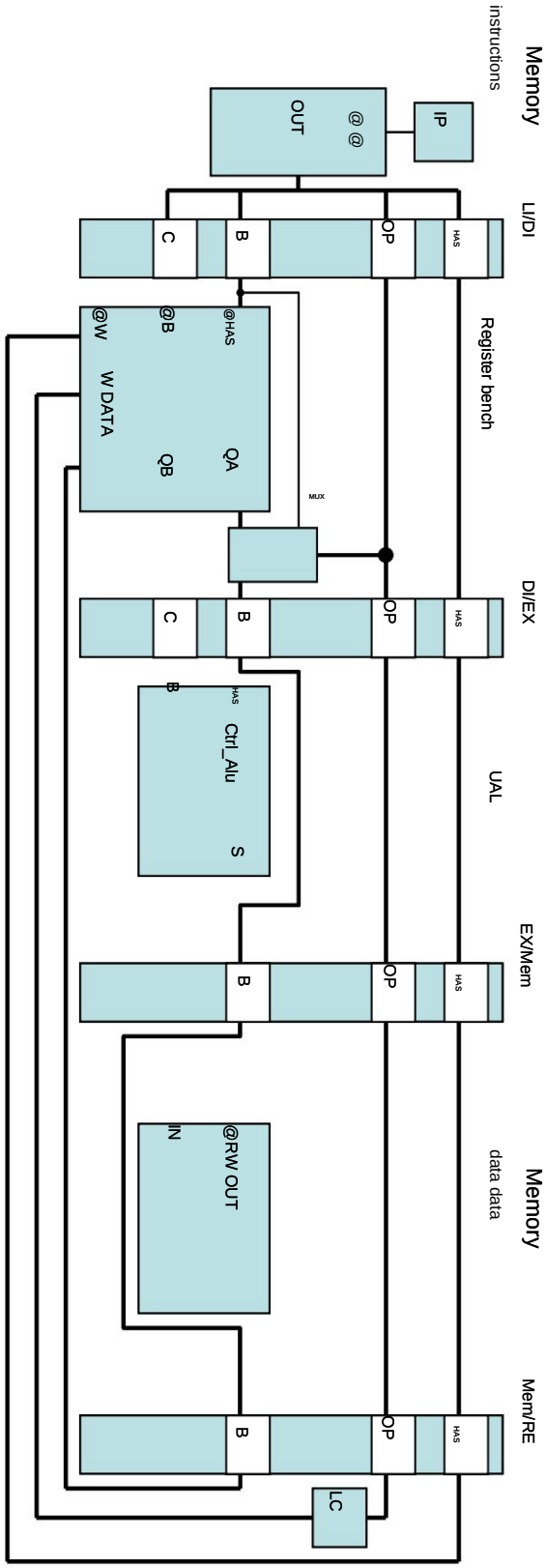


Figure 2: AFC, COP instructions

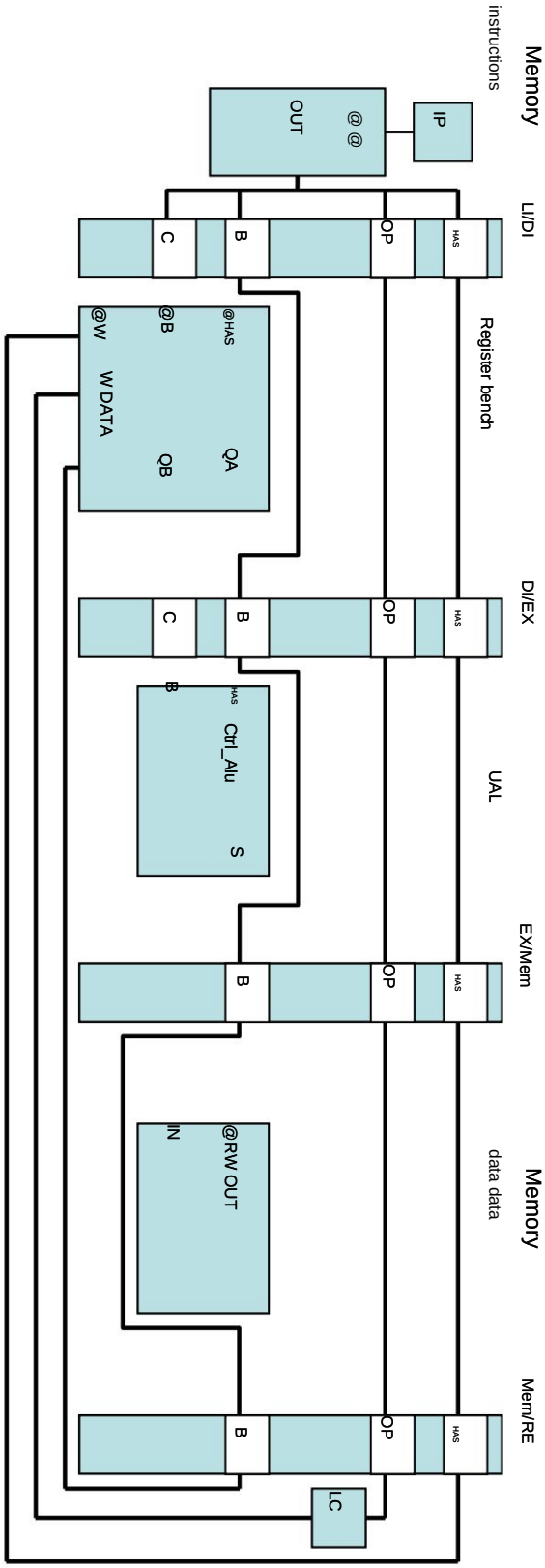


Figure 1: AFC instruction

Machine Translated by Google

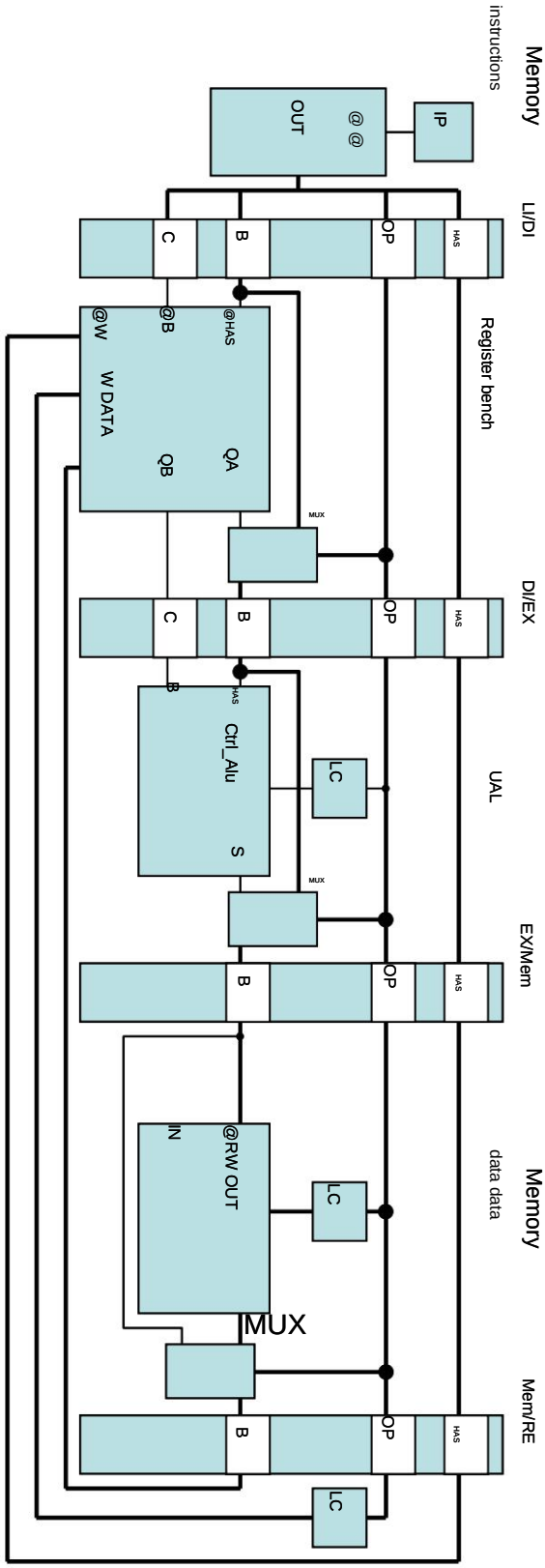


Illustration 4: Instructions AFC, COP, ADD, MUL, DIV, SOU, LOAD

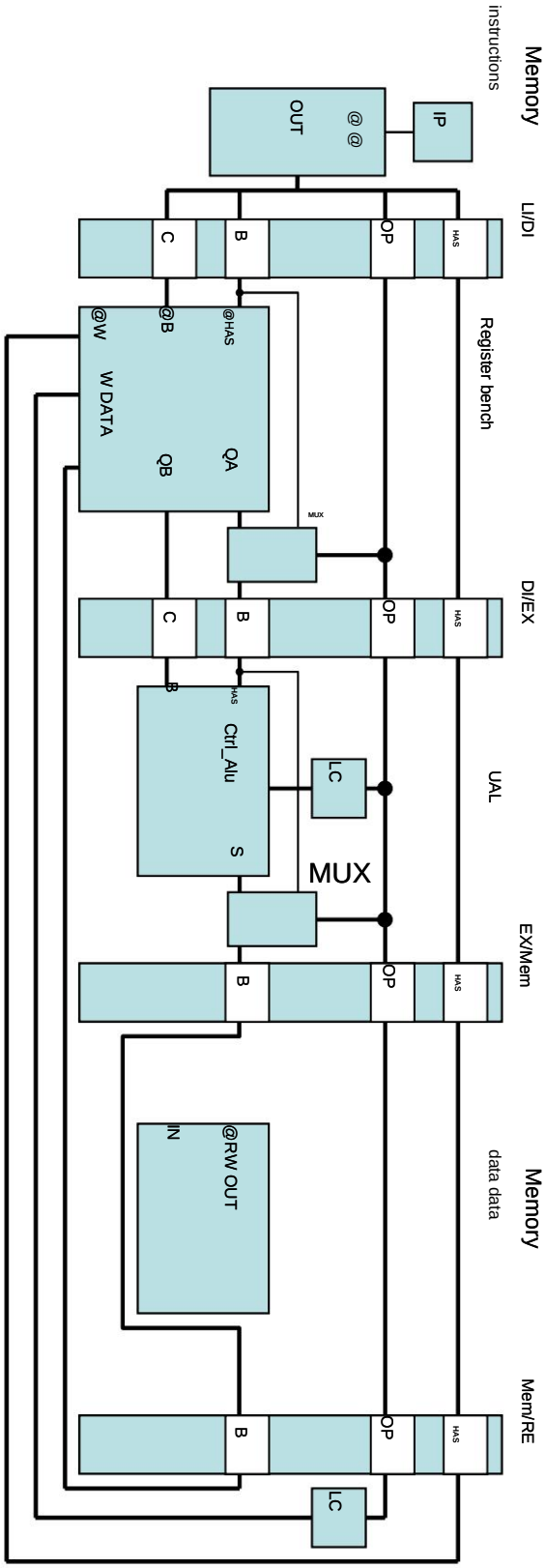


Illustration 3: Instructions AFC, COP, ADD, MUL, DIV, SOU

Machine Translated by Google

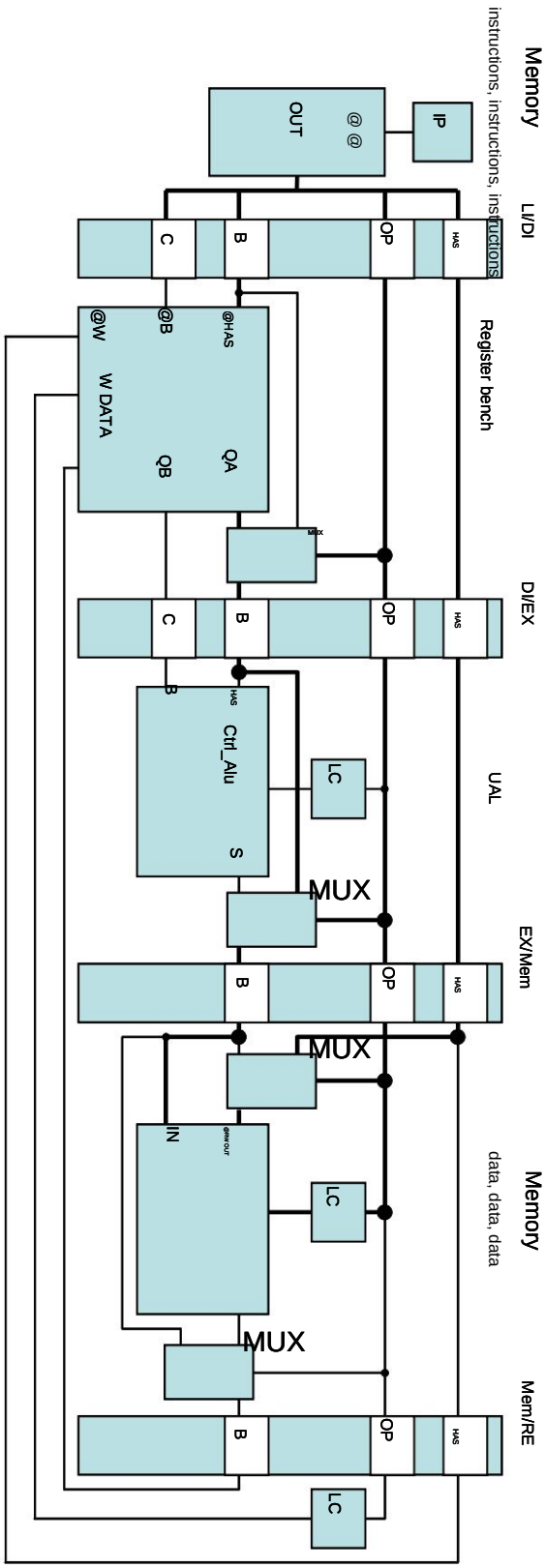


Illustration 5: Instructions AFC, COP, ADD, MUL, DIV, SOU, LOAD, STORE

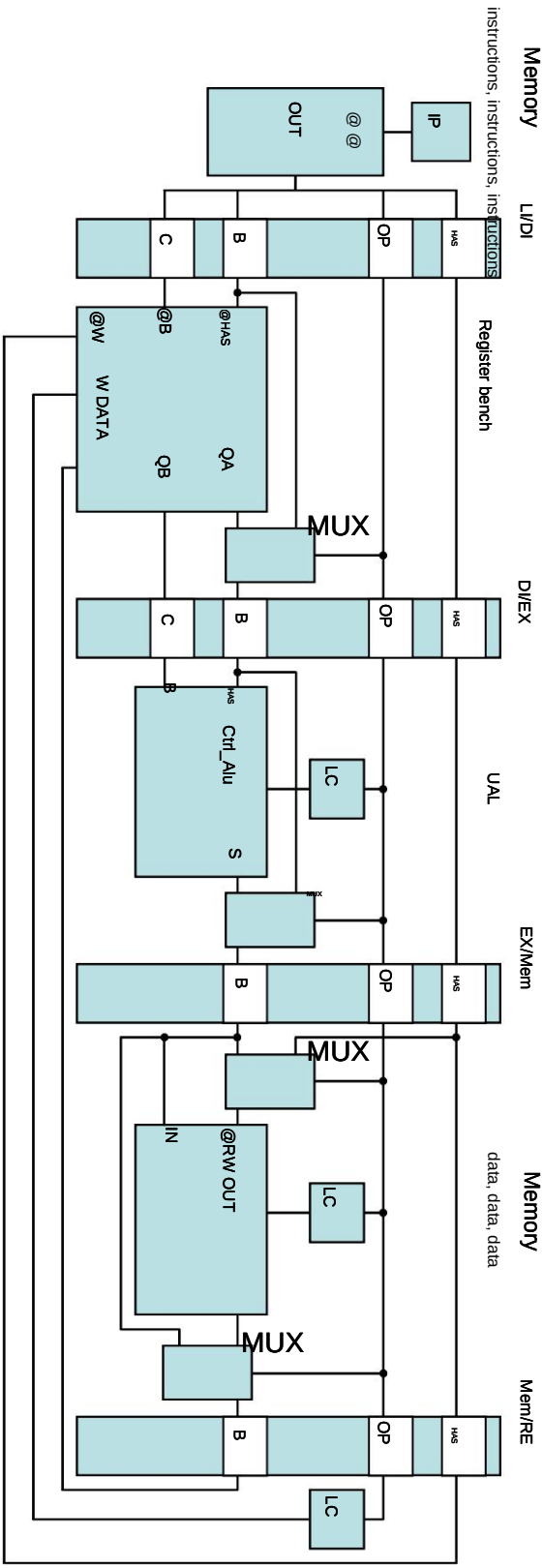


Illustration 6: Data path

Machine Translated by Google

Machine Translated by Google

Control unit

In addition to the data path, you will need to complete the control procedure, including the
Its role is to define the state of the control signals for the multiplexers and memory banks.
and other elements specific to each stage of the pipeline. This control is based on the instructions.
running.

Risk management – **optional in 2019-2020**

Subsequently, you will face unforeseen situations. There are two types of unforeseen events: hazards
data and branching hazards. We will focus on data hazards. In order to
To illustrate these situations, let us consider the following program:

```
…
NOP
NOP
[R1] ÿ 12
[R2] ÿ [R1]
NOP
NOP
…
```

Suppose that register *R1* contains *14* and register *R2* contains 8. At the end of this program,
The value assigned to register *R2* must be *12.* However, when the writing associated with the first
The instruction will be executed (stage 5), the value associated with register *R1* in stage 4 will not be
12 more , but *14.* Figure 9 illustrates this scenario. To solve this problem, we will...
We recommend using a risk management entity, whose role will be to inject "bubbles".
In case of detected hazards, these buffers allow for a delay in the execution of lower stages.
when a conflict with the upper floors is detected. For the previous example, when the
The first instruction is located at the level of the second pipeline – and the second instruction at the level
of the first pipeline – the comparison between the data from the first pipeline and the second pipeline
line allows us to detect this hazard:

Machine Translated by Google

OPDI_EX = AFC and OPLI_DI = COP and ADI_EX = BLI_DI

In this case, the clock of the first pipeline is inhibited while the first instruction finish. However, be careful that the "void" created by this delay does not alter the state registers. To avoid this, a *NOP (NO-OPERATION) must be injected,* which constitutes the bubble. The resolution of other hazard cases follows the same reasoning.

Machine Translated by Google

t	Entrance LI/DI	Entrance DI/EX	Entrance EX/Mem	Entrance Mem/RE	Entrance writing register	Contents of the register bench
10	OP = AFC A = R1 B = 12	NOP	NOP	NOP	NOP	R1 = 14; R2 = 8
10	OP = COP A = R2 B = R1	OP = AFC A = R1 B = 12	NOP	NOP	NOP	R1 = 14; R2 = 8
10	NOP	OP = COP A = R2 **B = 14**	OP = AFC A = R1 B = 12	NOP	NOP	R1 = 14; R2 = 8
10	NOP	NOP	OP = COP A = R2 B = 14	OP = AFC A = R1 B = 12	NOP	R1 = 14; R2 = 8
10	NOP	NOP	NOP	OP = COP A = R2 B = 14	OP = AFC A = R1 B = 12	**R1 = 12 ;** R2 = 8
10	NOP	NOP	NOP	NOP	OP = COP A = R2 B = 14	R1 = 12; **R2 = 14**

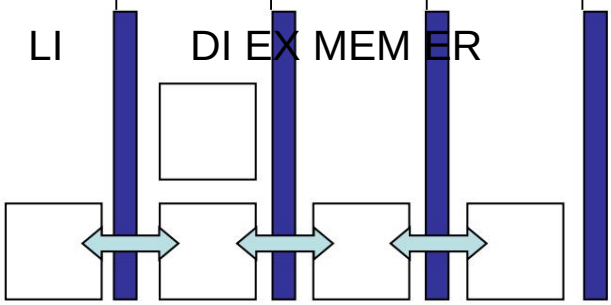


Figure 9: Example of data randomness